ALU Internship Connect — Final Flutter Project Technical Report

Author	Mildred Ewenrim Ebomah
Project	ALU Internship Connect (`launchpad_alu`)
Framework	Flutter (Dart 3, Material 3)
Backend	Firebase (Authentication, Cloud Firestore) + Cloudinary (media storage)
State Management	Riverpod 3
Navigation	go_router 17
Target Platform	Android (primary), iOS (compatible)
Codebase Size	73 Dart source files, ~6,900 lines of application code

1 Table of Contents

2 1. Project Overview

ALU Internship Connect is a cross-platform mobile application built for the African Leadership University (ALU) ecosystem. It connects two groups that need each other but currently have no dedicated channel to meet: **students seeking internship experience** and **student-led startups seeking talent**.

The application is deliberately *not* a job board. It is designed as an ecosystem for startup growth, experiential learning, and community building. It supports three user roles:

- **Students** — browse, search, filter, and bookmark opportunities; view startup profiles; apply with a resume, cover letter, and portfolio; track application status in real time; and receive in-app notifications.
- **Startups** — create an organization profile, post and manage internship opportunities, review incoming applicants, and update application statuses (which automatically notifies the student).
- **Administrators** — verify legitimate startups, granting them a trust badge visible throughout the app.

The project was delivered in six phases (setup/architecture → authentication/onboarding → startups/opportunities → applications/bookmarks/notifications → admin/security/polish → documentation and demo), all tracked in the repository README.

2.1 Key Functional Highlights

- Full email/password authentication with session persistence and password recovery.
- Role-based onboarding that branches into student or startup profile creation.
- Complete CRUD on opportunities (create, read, update, delete) restricted to the owning startup.
- Real-time synchronization everywhere: opportunity feeds, application lists, bookmarks, and notifications all use live Firestore streams — no manual refresh anywhere in the app.
- A five-stage application pipeline (Pending → Reviewed → Shortlisted → Accepted/Rejected) with automatic notification delivery on every transition.
- Admin verification workflow with a verified badge rendered on startup profiles and opportunity cards.

3 2. Problem Statement

Many ALU students struggle to obtain internship experience because established companies offer limited openings, hiring is highly competitive, and students often lack the prior professional experience those companies demand. At the same time, student entrepreneurs on campus need help across software development, UI/UX design, marketing, product management, operations, data analysis, finance, and content creation — but have no structured way to reach the student talent sitting in the same institution.

The two groups form a natural match: startups get motivated contributors at low cost; students get real-world, portfolio-building experience. What is missing is the **connective infrastructure**. ALU Internship Connect provides that infrastructure as a centralized, mobile-first platform where:

1. Startups publish structured opportunities (skills required, duration, work mode, deadline, openings, responsibilities, benefits).
2. Students discover those opportunities through search, filtering, and sorting, and apply directly in-app.

3 Both sides track the application lifecycle in real time, with trust established through an admin-driven verification system.

4 3. System Architecture

4.1 3.1 Architectural Style: Feature-First Clean Architecture

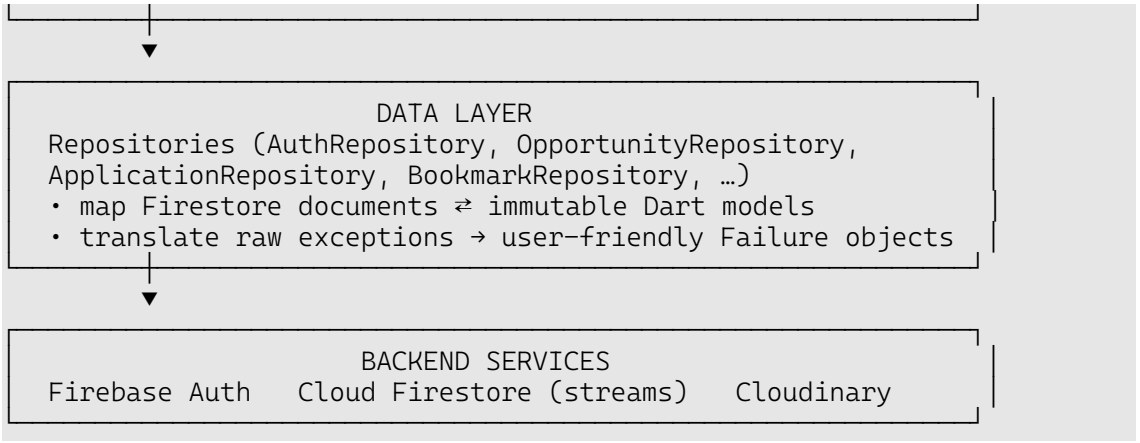
The codebase follows a **feature-first clean architecture**. Code is grouped by *business feature* rather than by technical type, and within each feature the data layer (models + repositories) is strictly separated from the presentation layer (screens + providers/controllers).

```
lib/
├── main.dart                  # Entry point: bootstraps Firebase,
mounts ProviderScope
├── bootstrap.dart             # Safe Firebase initialization with
failure capture
├── app.dart                   # Root MaterialApp.router with theming
├── firebase_options.dart      # Generated per-platform Firebase config
│
├── core/                      # Cross-cutting concerns (no feature
logic)
│   ├── constants/             # App constants, collection names,
UserRole enum
│   ├── config/                # Cloudinary configuration
│   ├── error/                 # Failure model + FirebaseErrorMapper
│   ├── providers/             # firebaseReadyProvider, repository
providers
│   ├── router/                # GoRouter setup + route path constants
│   ├── services/              # CloudinaryService (HTTP uploads)
│   ├── theme/                 # Material 3 light/dark themes, color
palette
│   └── utils/                 # Form validators
│
├── features/                  # One folder per business capability
│   ├── auth/                  #   data/ (AuthRepository) +
presentation/
│   ├── onboarding/            #   role-branched profile completion
│   ├── splash/                #   startup/loading screen
│   ├── home/                  #   HomeShell (bottom nav) + feed
│   ├── profile/               #   UserProfile model + UserRepository
│   ├── startup/               #   StartupProfile, dashboard, public
profile
│   ├── opportunities/         #   Opportunity model, repository, CRUD
screens
│   ├── applications/          #   Application model, apply/track/review
flows
│   ├── bookmarks/             #   BookmarkRepository + providers
│   ├── notifications/         #   AppNotification model + repository
│   └── admin/                 #   Startup verification screen
│
└── shared/
    └── widgets/               # 12 reusable widgets (cards, states,
inputs…)
```

4.2 3.2 Layered Data Flow

Every feature observes the same unidirectional flow. UI widgets never touch Firebase directly; all reads flow through streams, and all writes flow through controllers.

```
┌──────────────────────────────────────────────────────────┐
│                    PRESENTATION LAYER                    │
│  Screens (ConsumerWidget)  ←watch—  Providers (Stream/Async) │
│        │ user action                                     │
│        ▼                                                 │
│  Controllers (AsyncNotifier) — loading/success / error   │
```



Concretely, when a startup accepts an applicant:

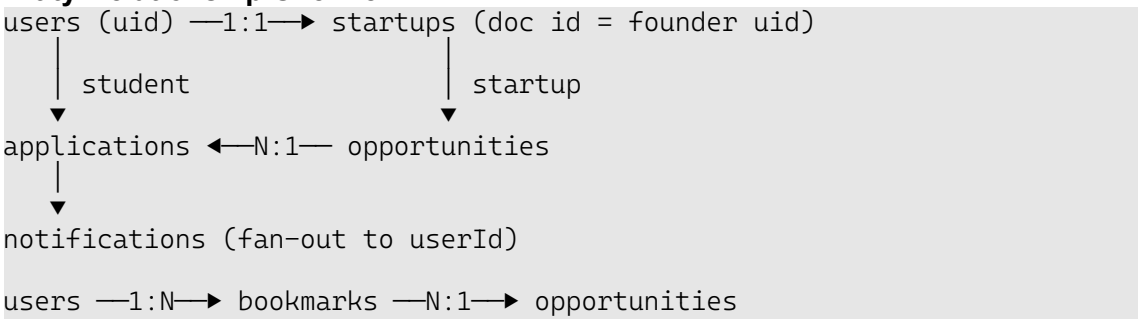
1. The screen calls `ApplicationStatusController.updateStatus(...)`.
2. The controller sets its state to `AsyncLoading`, then calls `ApplicationRepository.updateStatus(...)` which writes `{status, updatedAt}` with a merge-set.
3. The controller then calls `NotificationRepository.notifyApplicationStatusChange(...)`, which creates a notification document addressed to the student.
4. Firestore pushes the change through the active snapshot streams: the startup's applicant list, the student's "My Applications" screen, and the student's notification feed all update instantly — no polling, no refresh.

4.3 3.3 Resilient Bootstrap

`bootstrap.dart` wraps `Firebase.initializeApp` in a try/catch and returns a `BootstrapResult`. The result is injected into Riverpod as `firebaseReadyProvider` via a `ProviderScope` override in `main.dart`. Every repository provider checks this flag and returns `null` when Firebase is unavailable, and controllers surface a friendly network failure instead of crashing. This means the app **boots gracefully even if Firebase initialization fails** — a deliberate defensive design that also makes widget testing possible without a live backend.

5 4. Database Design

5.1 4.1 Entity-Relationship Overview



5.2 4.2 Firestore Collections

Six top-level collections are used. Collection names are centralized in `AppConstants` to eliminate typo-related bugs across repositories.

`users` — one document per authenticated user (document ID = Firebase Auth UID):

Field	Type	Notes
`role`	string	`student` | `startup` | `admin`
`email`	string	copied from Auth for querying/display
`fullName`, `program`, `year`	string?	student profile fields
`skills`, `interests`	array<string>	chip-input lists
`cvUrl`, `portfolioUrl`, `linkedInUrl`,	string?	Cloudinary/external URLs

Field	Type	Notes
`profileImageUrl`		
`onboardingComplete`	bool	drives the router's onboarding redirect
`createdAt`, `updatedAt`	timestamp	server timestamps

`startups` — one document per startup (document ID = founder's UID, giving a natural 1:1 link and simple security rules):

Field	Type	Notes
`founderId`, `founderName`	string	ownership + display
`name`, `description`, `industry`	string	organization profile
`teamSize`	int	
`logoUrl`, `website`, `contactEmail`, `contactPhone`	string?	optional profile data
`verified`	bool	set to `true` only by an admin

`opportunities` — internship/project postings:

Field	Type	Notes
`startupId`, `startupName`	string	denormalized owner info
`title`, `description`, `responsibilities`, `benefits`	string	posting content
`skillsRequired`	array<string>	used for skill filtering
`duration`, `industry`	string	filterable attributes
`internshipType`	string enum	internship/project / partTime / fullTime
`workMode`	string enum	remote / hybrid / onSite
`deadline`	timestamp	expired postings are hidden from the feed
`openings`	int	number of positions

`applications` — a student's application to one opportunity:

Field	Type	Notes
`opportunityId`, `opportunityTitle`	string	denormalized for list rendering
`startupId`, `startupName`	string	reviewer identity + rules
`studentId`, `studentName`, `studentEmail`	string	applicant identity
`status`	string enum	pending/reviewed/shortlisted/accepted / rejected
`resumeUrl`, `coverLetter`, `portfolioUrl`	string?	submission materials
`appliedAt`, `updatedAt`	timestamp	

`bookmarks` — lightweight join documents: `{userId, opportunityId, createdAt}`.

`notifications` — in-app inbox items: `{userId, title, body, type, read, relatedId, createdAt}`.

5.3 4.3 Design Decision: Strategic Denormalization

Firestore charges per document read and does not support server-side joins. The schema therefore **denormalizes display fields** (`startupName` on opportunities, `opportunityTitle`/`startupName`/`studentName` on applications) so that list screens render complete cards from a single query, with zero N+1 lookups. This trades a small amount of write-time duplication for dramatically cheaper and faster reads — the standard NoSQL pattern for read-heavy mobile apps (Firebase, 2024).

5.4 4.4 Composite Indexes

Five composite indexes are declared in `firebase/firestore.indexes.json` to support the app's filtered-and-ordered queries: applications by `studentId`, `startupId`, and `opportunityId` (each paired with `appliedAt` descending), opportunities by `startupId` + `createdAt`, and notifications by `userId` + `createdAt`.

5.5 4.5 Model Layer Conventions

All models are **immutable** (`const` constructors, `final` fields) with three standard members:

- `fromFirestore(DocumentSnapshot)` — a defensive factory that null-guards every field with sensible defaults, so a malformed document can never crash the UI.
- `toFirestore()` — trims strings, omits null optional fields, and always writes `updatedAt: FieldValue.serverTimestamp()` so ordering is consistent across devices with skewed clocks.
- `copyWith(...)` — supports immutable state updates.

Enumerated values (`UserRole`, `ApplicationStatus`, `InternshipType`, `WorkMode`, `NotificationType`) are stored as their Dart enum `name` string, each with a `fromName` parser that falls back to a safe default for forward compatibility.

6 5. Firebase Integration

6.1 5.1 Firebase Authentication

`AuthRepository` wraps `FirebaseAuth` behind a clean, injectable API (`signIn`, `register`, `sendPasswordReset`, `signOut`, `authStateChanges`). Session persistence and auto-login come from listening to `authStateChanges()` — the stream that Firebase persists across app restarts. On registration, the `RegisterController` performs a two-step transaction: create the Auth account, then immediately create the corresponding `users/{uid}` Firestore document with the selected role, so a user profile always exists before onboarding begins.

All raw `FirebaseAuthException` codes are translated by `FirebaseErrorMapper` into human-readable messages ("Incorrect email or password.", "An account already exists for that email.", "No internet connection. Please try again.") before they ever reach a widget.

6.2 5.2 Cloud Firestore: Real-Time by Default

Every list in the app is backed by a `snapshots()` stream, exposed through Riverpod `StreamProvider`s:

Stream	Consumers
`watchOpportunities(filters)`	home feed, opportunities browse screen
`watchOpportunitiesByStartup(id)`	startup dashboard
`watchStudentApplications(uid)`	student "My Applications" tracker
`watchStartupApplications(id)` / `watchOpportunityApplications(id)`	applicant review screens
`watchBookmarkedOpportunityIds(uid)`	bookmark toggles + saved list
`watchNotifications(uid)`	notification inbox + unread badge
`watchUnverifiedStartups()`	admin verification queue

Because the UI watches streams rather than one-shot futures, a status change made by a startup appears on the student's device within moments, satisfying the real-time requirement without any manual refresh mechanism.

6.3 5.3 Duplicate-Application Guard

Before submitting, `ApplyController` runs `hasApplied(studentId, opportunityId)` (a `limit(1)` query) and rejects duplicates with a clear failure message — enforcing one application per student per opportunity.

7 6. File Storage with Cloudinary

The project originally targeted Firebase Cloud Storage but was migrated to **Cloudinary** (Firebase Storage now requires a paid plan for new projects). The design keeps this swap

invisible to the rest of the app:

- `CloudinaryService` performs unsigned multipart uploads (`image` for profile photos and logos, `raw` for CVs/resumes) to folder paths namespaced under `launchpad_alu/` (`profile_images/`, `logos/`, `cvs/`, `resumes/{uid}`).
- Only the returned `secure_url` is persisted — **in Firestore** — so models, repositories, and UI code are entirely storage-provider-agnostic. Swapping providers again would touch exactly one service class.
- If Cloudinary is unconfigured or an upload fails, the service throws a typed `Failure` that surfaces as a normal error snackbar; profile flows degrade gracefully (e.g., an application can fall back to the CV already on the student's profile).

8 7. State Management

8.1 7.1 Why Riverpod

Riverpod 3 was chosen over Provider, BLoC, and setState-based approaches because it offers:

- **Compile-time safety** — providers are top-level globals; there is no `BuildContext` lookup that can fail at runtime.
- **First-class async primitives** — `StreamProvider` and `AsyncNotifier` model exactly the two state shapes this app needs (live Firestore data and imperative async actions), each with built-in `AsyncValue` loading/error/data handling.
- **Composability** — providers can watch other providers, which powers the app's dependency-injection story and reactive router.
- **Testability** — `ProviderScope(overrides: [...])` allows any dependency to be replaced in tests, which the widget test suite uses to boot the app with Firebase disabled.

8.2 7.2 The Three Provider Tiers

The app uses a consistent, layered provider vocabulary:

1. **Infrastructure providers** — `firebaseReadyProvider` (a boolean injected at startup) and repository providers (`opportunityRepositoryProvider`, `applicationRepositoryProvider`, …) that return `null` when Firebase is unavailable. This is constructor-free dependency injection: repositories can also accept a `FirebaseFirestore` instance for testing.
2. **Read providers** — `StreamProvider`s that expose repository streams to the UI (`authStateProvider`, `currentUserProfileProvider`, opportunity/application/bookmark/notification streams). Widgets `ref.watch(...)` these and pattern-match on `AsyncValue` to render loading, error, or data states.
3. **Action controllers** — `AsyncNotifier<void>` subclasses (`LoginController`, `RegisterController`, `ApplyController`, `ApplicationStatusController`, `AdminActionsController`, opportunity and onboarding controllers). Each action follows the same idiom:

```
state = const AsyncLoading();
state = await AsyncValue.guard(() async {
  // read repositories, validate, write to Firestore
});
```

`AsyncValue.guard` captures any thrown `Failure` into the error state, so every screen gets loading spinners and error snackbars from a single pattern with no bespoke try/catch code in widgets.

8.3 7.3 Efficient Rebuilds

Screens are `ConsumerWidget`s that watch only the providers they render. `select`-style granularity is achieved structurally: e.g., the bookmark state is a `Set<String>` of opportunity IDs, so toggling one bookmark rebuilds only the affected icon, and the `HomeShell` keeps all five

tabs alive in an `IndexedStack` so switching tabs never re-fetches data.

9 8. Navigation and Route Guarding

Navigation uses **go_router** with a single, reactive `routerProvider`. The router is itself a Riverpod provider that watches `authStateProvider` and `currentUserProfileProvider`, and a `refreshListenable` re-evaluates redirects whenever either stream emits.

The centralized `redirect` implements the entire access-control state machine:

```
auth loading?            → /splash
not logged in?           → /login  (register & forgot-password allowed)
profile loading?         → /splash
onboarding incomplete?   → /onboarding
logged in on auth route? → /home
otherwise                → allow navigation
```

This guarantees, in one auditable function, that an unauthenticated user can never reach an app screen, a half-onboarded user is always routed to complete their profile, and a logged-in user can never see the login screen again. Fifteen routes cover the full surface: splash, login, register, forgot-password, onboarding, home shell, opportunities (browse/create/detail/edit/apply), startup profile and dashboard, applications (list/detail), notifications, profile, and admin.

10 9. UI/UX Reasoning

10.1 9.1 Design System

- **Material 3** with a seeded `ColorScheme`, full **light and dark themes** following the system setting, and the **Inter** typeface (via `google_fonts`) for a modern, professional feel.
- All component styling (cards, inputs, buttons, chips, snackbars) is defined once in `AppTheme` with a consistent 14 px corner radius, so every screen is automatically consistent, and a redesign is a one-file change.
- Buttons use a full-width, 52 px height standard for comfortable touch targets.

10.2 9.2 Reusable Widget Library

Twelve shared widgets enforce visual and behavioral consistency: `AppTextField`, `AppPrimaryButton`, `AuthScaffold`, `ChipInputField` (skills/interests entry), `RoleSelectionCard`, `OpportunityCard`, `ApplicationStatusChip` (color-coded per status), `VerifiedBadge`, `AppLoading`, `AppEmptyState`, `AppErrorState`, and `AppSnackbar`. List screens therefore always express the full UX state grid — *loading → empty → error → data* — rather than showing a blank screen.

10.3 9.3 Interaction Patterns

- **Success feedback** via themed snackbars after every mutation (apply, post, verify, status change).
- **Confirmation dialogs** before destructive actions such as deleting an opportunity.
- **Empty states** with explanatory copy and calls-to-action (e.g., "No applications yet — browse opportunities").
- **Form validation** through centralized `Validators` (email format, password length, required fields, URL shape) applied on every input before any network call.
- **Role-adaptive UI** — the same shell serves both roles; students see apply/bookmark affordances while startups see a dashboard with posting management and applicant review.
- **Cached network images** (`cached_network_image`) for logos and avatars to avoid flicker and repeated downloads.

11 10. Security Considerations

Security is enforced **server-side** in `firebase/firestore.rules`, not merely in the UI:

Collection	Read	Create	Update	Delete
`users`	signed-in	owner only	owner or admin	admin
`startups`	public	founder only (`founderId == uid`)	founder or admin	admin
`opportunities`	public	owning startup only	owning startup	owning startup
`applications`	applicant, reviewed startup, or admin	student self only	startup or admin	admin
`bookmarks`	owner	owner	—	owner
`notifications`	recipient	signed-in	recipient (mark read)	admin

Key properties: students cannot edit startups or other users' data; startups cannot modify student profiles or forge applications; application privacy is preserved (only the applicant, the receiving startup, and admins can read one); and the `verified` flag is effectively admin-controlled through role-checked updates. Roles are resolved server-side by reading `users/{uid}.data.role` inside the rules, so a compromised client cannot self-elevate. Passwords are handled entirely by Firebase Authentication and never touch application code.

12 11. Error Handling Strategy

A single funnel handles every failure class:

1. Repositories catch raw exceptions and rethrow them as a typed `Failure(message, code)` via `FirebaseErrorMapper`, which maps auth codes and network conditions (`unavailable`, `network-request-failed`) to friendly copy.
2. Controllers wrap actions in `AsyncValue.guard`, converting thrown `Failure`s into `AsyncError` states.
3. Screens listen to controller state and show `AppSnackbar`/`AppErrorState` with a retry affordance.

Covered scenarios include: no internet, authentication failures, Firestore permission or availability errors, Cloudinary upload failures, empty search results, empty lists, duplicate applications, unconfigured backend (the `firebaseReady` flag), and client-side validation errors — every category the specification requires.

13 12. Testing

- **Widget testing**: the suite boots the entire app inside a `ProviderScope` with `firebaseReadyProvider` overridden to `false`, verifying that the splash→login redirect chain works and that the app renders without any live backend. This exercises the router, theming, and provider graph end-to-end.
- **Testability by design**: every repository accepts injected `FirebaseAuth`/`FirebaseFirestore` instances, and every dependency is a Riverpod provider that can be overridden — so unit tests can substitute fakes without touching production code.
- **Static analysis**: `flutter_lints` is enabled through `analysis_options.yaml`, and the codebase is fully null-safe.
- **Manual verification**: each phase was manually tested against its acceptance criteria on an Android device/emulator — both role journeys end-to-end (register → onboard → post/browse → apply → review → status change → notification), plus offline behavior and validation edge cases.

14 13. Performance and Scalability

Implemented optimizations

- **Denormalized reads** — every list screen renders from one query (no join fan-out).
- **Composite indexes** — all filtered/ordered queries are index-backed.
- **Server timestamps** — consistent ordering regardless of device clock skew.
- `IndexedStack` **tab shell** — streams stay warm across tab switches; no re-fetch churn.
- **Image caching** — network images are disk-cached.
- **Expired-post filtering** — the feed hides past-deadline opportunities automatically.
- **Batched writes** — "mark all notifications read" commits one Firestore batch instead of N writes.

Scalability posture

The architecture scales horizontally with Firestore. Adding a feature means adding a folder under `features/` with its own models, repository, providers, and screens — no existing code needs modification. Two known trade-offs are documented honestly: (1) opportunity search/filtering currently happens client-side on the streamed feed, which is ideal for a campus-scale corpus (hundreds of posts) but should move to indexed Firestore queries or a search service (e.g., Algolia) at larger scale; and (2) pagination scaffolding exists (`AppConstants.pageSize`) and cursor-based `startAfter` pagination is the designated next step once feed volume warrants it.

15 14. Challenges and Solutions

#	Challenge	Solution
1	**Firebase Storage requires a paid plan**, blocking file uploads on the free tier.	Migrated to Cloudinary unsigned uploads; stored only `secure_url`s in Firestore behind a single service class, keeping the rest of the codebase provider-agnostic.
2	**Auth/onboarding redirect loops** — routing depends on two async sources (auth state and profile document) that resolve at different times.	Modeled the guard as an explicit state machine in one `redirect` function, holding users on `/splash` until both streams settle, with a `refreshListenable` re-running redirects on every emission.
3	**Firestore's query limitations** — no joins, no `OR` filters, composite-index requirements for multi-field queries.	Strategic denormalization for display fields; declared composite indexes in version control; combined a single indexed server query with client-side refinement for the multi-facet filter UI.
4	**Keeping both parties in sync on application status** without push infrastructure.	Firestore snapshot streams on both sides plus an in-app notifications collection written transactionally alongside every status change.
5	**Graceful degradation when the backend is unavailable** (offline devices, misconfiguration, CI).	The `BootstrapResult`/`firebaseReadyProvider` pattern: repositories become `null`, controllers raise friendly network failures, and the app remains navigable rather than crashing.
6	**Preventing duplicate or forged applications.**	Client-side `hasApplied` guard for UX, backed by server-side security rules that require `studentId == request.auth.uid` on creation.
7	**Consistent error UX across ~15 screens.**	Centralized `FirebaseErrorMapper` + `Failure` type + the `AsyncValue.guard`

#	Challenge	Solution
		controller idiom, so no widget contains ad-hoc error handling.

16 15. Future Improvements

1. **Push notifications** via Firebase Cloud Messaging, complementing the in-app inbox with lock-screen delivery.
2. **In-app messaging** between startups and applicants (the `messages` collection name is already reserved in `AppConstants`).
3. **Cursor-based pagination** on the opportunity feed as content volume grows.
4. **Server-side search** (Algolia or Firestore full-text extensions) replacing client-side filtering at scale.
5. **Recommendation engine** ranking opportunities by overlap between a student's skills/interests and posting requirements.
6. **Startup analytics dashboard** — applications received, views per posting, conversion funnel.
7. **Interview scheduling** with calendar integration and automated reminders.
8. **Expanded automated testing** — repository unit tests against the Firestore emulator, golden tests for shared widgets, and integration tests for the two core user journeys.
9. **CI/CD** — GitHub Actions running analyze/test on every push, with signed release builds (Android signing configuration is already in place per `ANDROID_RELEASE.md`).

17 16. Screenshots

Screenshots of the running application accompany this report (see `UI_SCREENSHOTS.md` in the repository for the capture checklist). The demo video walks through each flow live.

Suggested figure list:

1. Login and registration (role selection)
2. Student onboarding (skills/interests chip input, CV upload)
3. Startup onboarding (organization profile, logo upload)
4. Home feed and opportunity browse with search/filters
5. Opportunity detail with bookmark and apply actions
6. Application form and "My Applications" tracker with status chips
7. Startup dashboard: postings and applicant review
8. Notifications inbox (real-time status updates)
9. Admin verification queue and verified badge
10. Dark mode rendering of the feed

18 17. Conclusion

ALU Internship Connect delivers the full required feature set — Firebase email/password authentication with session persistence, role-based onboarding, startup profiles with verification, complete opportunity CRUD, discovery with search/filter/sort, a five-stage application pipeline, bookmarks, and real-time in-app notifications — on a clean, feature-first architecture with Riverpod state management, guarded go_router navigation, and server-enforced Firestore security rules. The codebase (73 Dart files, ~6,900 lines) demonstrates production-quality patterns: immutable models, repository abstraction, centralized error mapping, a reusable widget system covering every UX state, and defensive bootstrapping that keeps the app functional even when the backend is unreachable. The result is a maintainable, scalable foundation that solves a genuine problem in the ALU ecosystem and is ready to grow into messaging, recommendations, and analytics.

19 18. References

Dart Team. (2025). *Effective Dart: Style, documentation, and design guidelines*. Google. https://dart.dev/effective-dart

Firebase. (2024). *Cloud Firestore data model and best practices*. Google. https://firebase.google.com/docs/firestore/data-model

Firebase. (2024). *Firebase Authentication documentation*. Google. https://firebase.google.com/docs/auth

Firebase. (2024). *Get to know Cloud Firestore security rules*. Google. https://firebase.google.com/docs/firestore/security/get-started

Flutter Team. (2025). *Flutter documentation: Architecture and state management*. Google. https://docs.flutter.dev

FlutterFire. (2025). *FlutterFire: Firebase plugins for Flutter*. Invertase. https://firebase.flutter.dev

Material Design Team. (2024). *Material Design 3 guidelines*. Google. https://m3.material.io

Remi Rousselet. (2025). *Riverpod documentation* (Version 3). https://riverpod.dev

Cloudinary. (2025). *Upload API reference: Unsigned uploads*. https://cloudinary.com/documentation/upload_images

go_router contributors. (2025). *go_router package documentation* (Version 17). Flutter Community. https://pub.dev/packages/go_router